

Solving and Generating Sudoku as a Constraint Satisfaction Problem

CS 175: Project in AI. Final Report. Group 3

Arjun Mohunta, amohunta, 80654131

Haoyang Ding, haoyad6, 44024972

Rajat Choudhary, rajatc1, 37222868

Source code: <https://github.com/arjunmohunta/suduko>

1. Project Summary

A Sudoku puzzle is a 9x9 grid, partially filled, that must be completed so that every row, every column, and every 3x3 box contains each of the digits 1 through 9 exactly once. Our system does two things with such grids: given any valid partial grid it returns the completed solution together with statistics describing the search it performed, and given a requested difficulty level it generates a new puzzle verified to have exactly one solution.

Formally this is a constraint satisfaction problem. The variables are the 81 cells, each with domain $\{1, \dots, 9\}$, and the constraints are 27 all-different constraints, one per row, column, and box. Because a well-formed Sudoku has a unique solution, the task is not to find a consistent assignment but the consistent assignment.

Why the problem is not trivial. The assignment space for a 17-clue puzzle is 9^{64} , about 10^{61} , so no amount of hardware enumerates it. What makes Sudoku tractable is not speed but inference. The constraints are dense, since every cell shares a constraint with exactly 20 others, so each assignment carries information about a fifth of the grid. A solver that exploits this searches a vanishingly small fraction of the space, and one that ignores it does not finish. Section 3.2 measures that gap: on a 17-clue puzzle our baseline expands more than ten million nodes without solving, while a solver that propagates constraints finishes after expanding one node.

Why AI methods are required. Sudoku is a puzzle in the classical AI sense: no environmental feedback, no reward signal, no training distribution. Success comes from search and logical inference over a declarative constraint model, so the classical CSP methods apply rather than learning-based ones. We use no machine learning, and this is a modelling decision rather than an omission: the constraints are known exactly and are hard, so an inference procedure can guarantee correctness and uniqueness, which a learned approximator cannot. The same all-different structure appears in timetabling, shift rostering, and register allocation.

2. System Architecture and Algorithms

2.1 Structure, inputs and outputs

The system is one CSP core in `sudoku.py` with four consumers: a command-line demo (`demo.py`), a browser demo (`index.html`), an evaluation harness (`benchmark.py`), and a

regression suite (`tests.py`). Nothing in the core knows about presentation, and nothing in the interfaces re-implements reasoning.

A board is a list of 81 integers in row-major order, 0 for empty. `parse` accepts an 81-character string or a file path, with either 0 or a period as the blank marker, and `load_many` reads the multi-puzzle benchmark files. Every solver returns a solution or `None` plus a `Stats` record carrying solved, backtracks, nodes, `time_ms`, and capped. The generator returns the puzzle, its solution, and the clue count.

2.2 Representation

Two choices do most of the performance work. First, domains are bit masks: the candidate set of each cell is a 9-bit integer, so removing a value is one bitwise AND, the domain size that MRV needs at every node is a precomputed population count lookup rather than a loop, and a domain wipeout is exactly a mask equal to zero. Second, the constraint scopes are precomputed once at import: `PEERS[i]` holds the 20 cells sharing a row, column, or box with cell `i`, and `UNITS` holds the 27 nine-cell scopes. This is why propagation is worth running on every assignment: a single assignment prunes 20 domains at once.

2.3 Solver 1: plain backtracking (the baseline)

Find the first empty cell in row-major order, try digits 1 through 9, test each against the current row, column, and box, and recurse on the first that fits. No domains, no inference, no ordering. A node cap ensures an intractable instance terminates rather than hanging. Hitting the cap is reported as giving up, and because the run did not finish, any ratio computed against a capped run is a lower bound and never a measured value. This applies everywhere the baseline appears below. Our in-class presentation used a 400,000-node cap so that the comparison finished quickly during the demo; for this report we raised it to 10,000,000, which is the figure our progress report quoted, so the failure of the baseline is demonstrated against a far larger budget.

The baseline is obviously correct and untuned, which makes it a trustworthy control: the only difference from Solver 2 is forward checking and MRV, so the comparison isolates what those two techniques contribute. Its weakness is that it detects a contradiction only on reaching the offending cell, so it re-explores subtrees the constraints had already excluded.

2.4 Solver 2: forward checking and MRV

Forward checking. The moment value `v` is assigned to cell `i`, remove `v` from the domain of every unassigned peer `p`:

```
for each p in PEERS(i):    D_p := D_p \ {v}
```

If this leaves any `D_p` empty the assignment cannot lead to a solution, so we undo it immediately rather than discovering the failure deeper in the tree.

Minimum Remaining Values. Branch on the unassigned cell with the smallest domain:

```
i* = argmin |D_i|    over all cells i with no value yet
```

This is the fail-first principle: a cell with one candidate is filled without guessing, a cell with zero exposes the contradiction now, and otherwise the search branches where the branching factor is smallest.

```
function SOLVE-SMART(board):
  cand <- initial domains derived from the givens
  function BT():
    i <- unassigned cell with the smallest domain
    if there is none: return success
    for each v in D_i:
      val[i] <- v
      remove v from D_p for every p in PEERS(i), recording each change
      if no domain was emptied and BT() succeeds: return success
      undo val[i] and restore every recorded domain
    return failure
  return BT()
```

Undo is exact, since the domains actually modified are recorded per branch and restored on backtrack. The cost per node is very low, one bitwise pass over 20 peers, and this is already enough to solve an easy puzzle by propagation alone. Two weaknesses remain. MRV rescans all 81 cells at every node, an $O(n)$ cost where an incremental structure would be $O(\log n)$. More importantly, forward checking reasons only outward from an assignment to its peers, which leaves the class of inference in Section 2.5 unused. We did not add Least Constraining Value, which Russell and Norvig pair with MRV, because forward checking already prunes early enough that value order had no measurable effect on our instances.

2.5 Solver 3: adding hidden singles

This is the stronger propagation that our progress report listed as a stretch goal. Forward checking is cell-centred, reasoning from an assignment outward to the peers it constrains. The complementary inference is unit-centred, and both forms below are run to a fixpoint before every branching decision.

- Naked single: a cell whose domain has one value left must take it.
- Hidden single: if, within one of the 27 units, a value can be placed in only one cell, it must go there. This holds even when that cell still has several candidates, which is exactly why forward checking cannot see it.

```
function PROPAGATE(val, cand):
  repeat until nothing changes:
    for each unassigned cell i:
      if D_i is empty:      return contradiction
      if |D_i| = 1:         ASSIGN(i, the one remaining value)
    for each unit U, for each value v not yet placed in U:
      spots <- cells i in U that are empty and still allow v
      if spots is empty:    return contradiction
      if |spots| = 1:       ASSIGN(that cell, v)
  return consistent
```

Solver 3 then runs the same MRV search as Solver 2, except that it assigns the givens and propagates first, and at every node it copies the domains, assigns, and propagates again before recursing. Note the second contradiction test: a unit in which some value has no remaining home is inconsistent even when no individual domain is empty, and forward checking never

detects this. Copying domains per branch rather than undoing from a trail costs 81 integers per node and buys a simpler correctness argument, since there is no undo path to get wrong.

The gain is dramatically less search, and on our presets it eliminates search entirely. The cost is that each node is far more expensive, a fixpoint over 27 units by 9 values instead of one pass over 20 peers, so where Solver 2 was already fast the extra inference buys nothing. Section 3.3 quantifies both sides.

2.6 Uniqueness testing, generation, and difficulty rating

`count_solutions(board, limit)` is the same MRV search, continuing past the first solution and stopping once limit solutions are found. Calling it with limit 2 answers whether a puzzle is unique for roughly the cost of one solve, since it aborts the moment a second solution appears. The generator uses it to guarantee uniqueness by construction, so no post-hoc filtering is needed and no multi-solution puzzle can escape.

```
function GENERATE(level, seed):
  solution <- SOLVE-SMART(empty grid, random value order)
  puzzle <- solution
  for each cell position p in random order:
    if the clue count has reached the target for level: stop
    saved <- puzzle[p]; puzzle[p] <- 0
    if COUNT-SOLUTIONS(puzzle, 2) is not 1:
      puzzle[p] <- saved      (the removal broke uniqueness, so undo it)
  return puzzle, solution, clue count
```

Targets are 45 clues for easy, 34 for medium, and 26 for hard, and seeding makes generation exactly reproducible. Generation is the most expensive operation in the system, since each removal costs a uniqueness check, and the result is irreducible only with respect to the random removal order rather than globally minimal.

Difficulty rating. Our proposal promised a rating and our progress report conceded that clue count is a weak proxy. `rate_difficulty` therefore rates by the search effort the reference solver actually needs: trivial at 0 backtracks, easy at 10 or fewer, medium at 100 or fewer, hard at 1000 or fewer, extreme above that. Section 3.5 shows why.

2.7 Visualisation

`solve_smart(record=True)` emits an event trace, recording a selection event when MRV picks a cell, a placement event on assignment, and an undo event on backtrack. Both front ends replay it, `demo.py animate` in the terminal and `index.html` in the browser with live candidate pencil marks.

3. Evaluation

3.1 Experimental setup

Our primary metric is search effort, meaning nodes expanded and backtracks performed. It directly reflects what the AI techniques do, since a heuristic earns its place by pruning the search space, and it is exactly reproducible because the search is deterministic. Wall-clock time is secondary and should be read as indicative only: our measurements were taken on a loaded

shared machine where repeated runs of the same deterministic solve varied by up to a factor of 8 while node counts did not vary at all.

A solve counts as correct only if the returned grid is complete, violates no all-different constraint, and preserves every given. Every solve rate below is verified this way rather than taken from the success flag of the solver. Large sets are sampled with a fixed seed rather than truncated, so a subset is representative and reproducible.

Benchmark set	Puzzles	Character
kaggle100k	100,000	export of the Kaggle 1 million Sudoku games set
minimal17	49,158	minimal 17-clue puzzles, the proven minimum
top1465	1,465	hard list from magictour
forum_hardest1106	375	forum-curated for difficulty against solvers
top95	95	hard 95 collected by Norvig
hardest11	11	hardest set collected by Norvig

3.2 Do the CSP techniques work?

The three presets give the like-for-like comparison, since the only differences between the solvers are the inference techniques.

Puzzle	Clues	Baseline backtracks / nodes	Solver 2 backtracks / nodes	Solver 3 nodes	Node ratio
easy	30	4,157 / 4,209	0 / 52	1	81x
AI Escargot	23	8,911 / 8,970	151 / 210	11	43x
extreme	17	hit the 10M cap	4,048 / 4,113	1	more than 2,431x

Propagation alone can be enough. On the easy puzzle, forward checking with MRV never guesses: every node is forced. The solver is reasoning, not searching.

The baseline does not merely lose, it fails. On the 17-clue puzzle it exhausted a ten million node budget without finding a solution, while the informed solvers finish the same puzzle in milliseconds. That is not a constant-factor speedup but the difference between a solver that terminates and one that does not.

The 17-clue puzzle also requires no search at all under stronger propagation, since the fixpoint of naked and hidden singles solves it outright. The puzzle was engineered to defeat chronological backtracking, and it does, but its difficulty is an artefact of weak inference rather than a property of the instance. On AI Escargot, the named hard instance from our proposal, Solver 2 finishes in 0.63 ms, so the moonshot we stated of solving the hardest known puzzles in well under a second is met by roughly three orders of magnitude.

3.3 Stronger propagation across the benchmark sets: the crossover

Running both informed solvers over samples from all six sets shows that node savings are universal but time savings are not. Both returned the identical verified solution on every puzzle, zero disagreements, which is our correctness check on the new solver as much as a performance result.

Set	n	Solver 2 mean nodes	Solver 3 mean nodes	Reduction	Time ratio
minimal17	150	50,880.0	4.0	12,827x	0.01x
top95	95	15,722.7	34.6	455x	0.15x
top1465	150	6,520.6	34.8	188x	0.46x
forum1106	150	15,053.0	150.8	100x	0.76x
hardest11	11	604.5	6.5	94x	0.87x
kaggle	150	48.2	1.0	48x	1.45x

Where it wins, it wins enormously. On the 17-clue set the worst case falls from 666,020 nodes to 46, 40.7 percent of those puzzles are solved with no search at all, and the time ratio of 0.01 is a speedup of about 150 times.

Where it loses, it loses for a comprehensible reason. On kaggle the extra inference is 45 percent slower despite expanding 48 times fewer nodes, because those puzzles were already solved with zero backtracks by forward checking alone. There was no search left to prune, so the fixpoint is pure overhead. This generalises past Sudoku: the value of an inference technique depends on how much search it prevents, not how much it deduces. Neither solver dominates, and the right default depends on the workload.

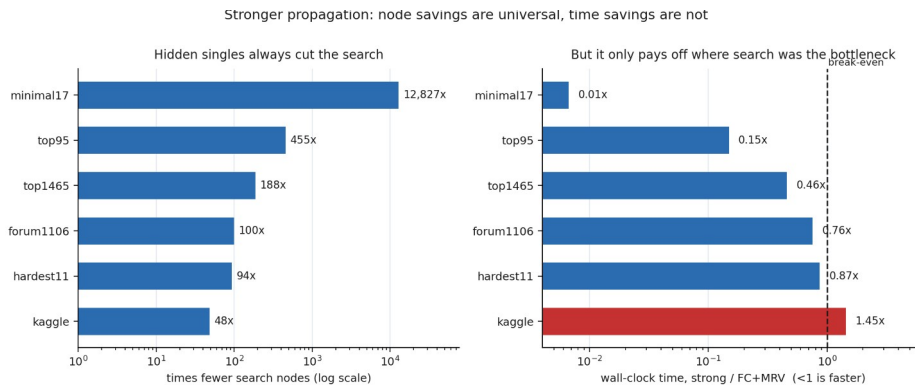


Figure 1. Stronger propagation cuts nodes on every set, but saves wall-clock time only where search was actually the bottleneck.

3.4 Solve rate over the full benchmark sets

Benchmark set	n	Solve rate	Median backtracks	Worst case
kaggle	2,000	100.00%	0	0
hardest11	11	100.00%	151	3,610
top1465	1,465	100.00%	1,441	268,371

forum1106	375	100.00%	10,690	84,896
top95	95	100.00%	2,424	268,371
minimal17	2,000	100.00%	13,920	2,460,815

Solve rate is 100 percent on all six sets, 5,946 puzzles with zero failures. No puzzle in any public set we tested defeats the solver, and this is now supported by a sample four orders of magnitude larger than the three puzzles in our progress report.

Two features of the table matter more than the headline. On kaggle the worst case is zero backtracks, not merely a low mean, so for the entire distribution of puzzles a human would encounter the search component of the search algorithm never runs. On minimal17 the worst case is 177 times the median within a set where every puzzle has exactly 17 clues, which is why every row carries a median and a maximum rather than a mean. The same baseline, capped at 200,000 nodes over a 50-puzzle subsample of each set, solves 100 percent of kaggle but only 42.0 percent of top1465, 40.0 percent of top95, 24.0 percent of forum1106, and 2.0 percent of minimal17, so its failure rate tracks difficulty exactly.

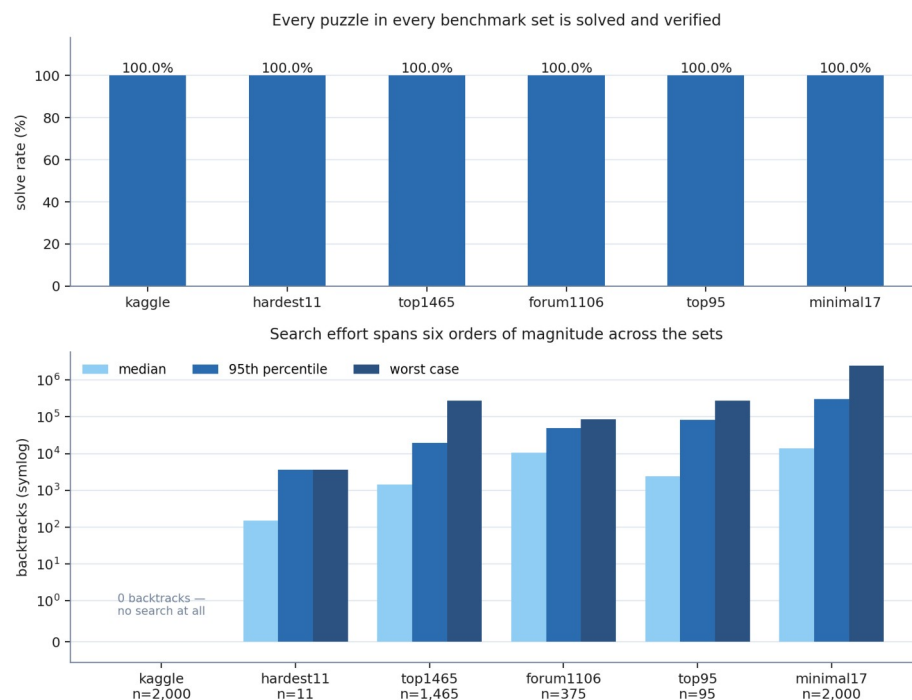


Figure 2. Solve rate and the distribution of search effort across all six benchmark sets.

3.5 Generator calibration

Level	Target clues	Unique solution	Needed no search	Median backtracks	Max
easy	45	30/30	30/30	0	0
medium	34	30/30	27/30	0	12
hard	26	30/30	4/30	32.5	277

--	--	--	--	--	--

Uniqueness is exact: 90 of 90 generated puzzles have precisely one solution. Clue count separates the levels perfectly, and difficulty only partly, since the effort distributions overlap and four of the 30 hard puzzles need no backtracking at all. Figure 3 makes the same point at larger scale on the public data: pooling all 5,946 puzzles by clue count, mean effort falls from 69,425 backtracks at 17 clues to 569 at 26, then to exactly zero for all 1,998 puzzles with 31 or more clues, yet at 17 clues alone the 95th percentile is 22 times the median. This is why `rate_difficulty` rates by measured effort.

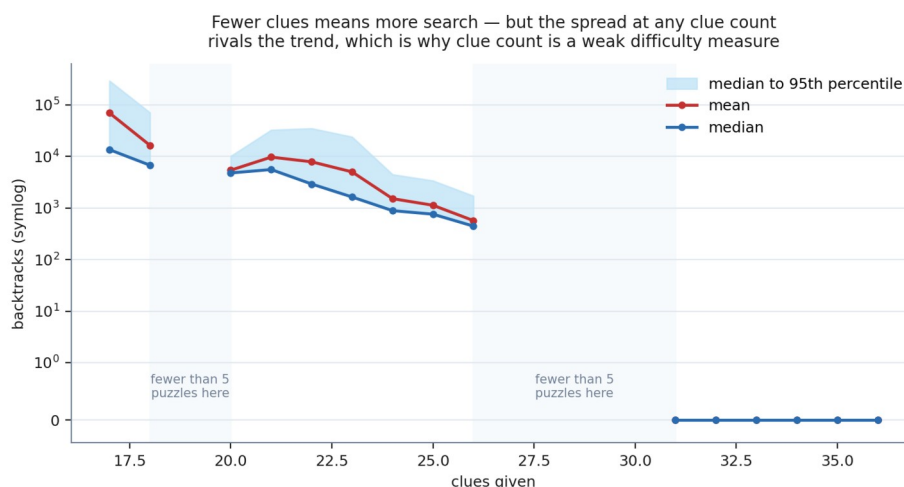


Figure 3. Search effort against clue count, pooled over all 5,946 puzzles.

3.6 Qualitative evaluation and limitations

Visualisation. Because the solver emits its own decision trace, a viewer watches the search that produced the numbers above rather than a reconstruction. On an easy puzzle the MRV highlight moves from forced cell to forced cell and no cell is ever un-filled, which is what zero backtracks looks like. On AI Escargot the highlight settles on a cell with two or three candidates, peers are pruned, and after a few levels the trace unwinds in red as the branch dies, which is what search looks like when propagation is insufficient.

Sanity checks, all automated and all passing. An already solved grid returns in 1 node with 0 backtracks. An easy puzzle solves with no backtracking. An empty grid is reported as having multiple solutions rather than one. A grid containing duplicate givens is flagged before solving is attempted. A grid that is legal but unsatisfiable returns no solution rather than an invalid one.

Regression suite. 24 automated tests cover peer symmetry, parsing, all three solvers, the node cap, the uniqueness counter, clue targets, seed reproducibility, and the difficulty rating. The most important is the agreement test, which checks that Solver 2 and Solver 3 return the identical grid on 40 hard puzzles, because a faster solver that returns a different grid is not a faster solver but a broken one.

Limitations. Timing is measured on a loaded machine and is indicative only, though node counts are exact. Large sets are sampled rather than exhausted, so a rare pathological instance in the remainder cannot be ruled out, and the three-way solver comparison uses smaller samples than the solve-rate experiment. Our difficulty rating measures machine difficulty and was not validated against human solving times, and generated puzzles are irreducible only with respect to the random removal order. Propagation stops at hidden singles: we do not implement pairs and triples, box-line reduction, X-wing, full arc consistency, or an exact-cover encoding, which is how the fastest published solvers work.

4. References and Resources Used

1. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach. The constraint satisfaction chapter, for backtracking search, forward checking, and MRV.
2. P. Norvig, Solving Every Sudoku Puzzle, <https://norvig.com/sudoku.html>. This informed our bitmask candidate representation, peer-based pruning, and our use of the naked and hidden single pair as the propagation fixpoint. No code was copied.
3. Course materials for CS 175, for the framing of puzzles as tasks with no environmental feedback.
4. Benchmark data, all public and none authored by us. top95 and hardest11 collected by Peter Norvig, <https://norvig.com/top95.txt> and <https://norvig.com/hardest.txt>. kaggle100k, a 100,000-puzzle export of the Kaggle 1 million Sudoku games dataset, originally <https://www.kaggle.com/bryanpark/sudoku>. minimal17, 49,158 minimal 17-clue puzzles from the collection of 17-clue grids by Gordon Royle and later extensions. top1465 from <http://magictour.free.fr/sudoku.htm>. forum_hardest1106, a forum-curated collection selected for difficulty against solvers. The last four sets are distributed together in the data archive of the tdoku Sudoku solver benchmark project by T. Dillon, <https://t-dillon.github.io/tdoku>, and our `fetch_data.py` downloads that archive and extracts them, so the data step of our evaluation is reproducible by running one script.
5. AI Escargot, by Arto Inkala, published 2006, our named hard single instance. The AI in the name stands for the initials of the author, not artificial intelligence.
6. Software: the Python 3 standard library only for the solvers, generator, and command-line tool, with no third-party AI or machine learning packages anywhere in the system. The standard `unittest` module for the regression suite, and `matplotlib` for the figures, used only in `figures.py`, which reads the stored results and draws them so that no figure recomputes a value.

Reproducing this report. Run `python3 fetch_data.py`, `python3 tests.py`, `python3 benchmark.py all`, then `python3 figures.py`. Node and backtrack counts reproduce exactly; timings depend on the machine.

Statement of original work. The CSP formulation, all three solvers, the uniqueness counter, the generator, the difficulty rating, both visualisations, the test suite, and the evaluation harness are our own work for this course. The algorithms are standard and are credited above, and the puzzle data is third-party and public as itemised in item 4.